학습 노트: LangSmith Evaluation Harness (LLM 평가 자동화)

작성일: 2026-05-24
관련 파일: [dev2/tests/eval/golden_dataset.py], [dev2/tests/eval/test_severity_eval.py]
복습 예정: D+1, D+3, D+7

한 줄 요약

"AI가 매긴 점수가 정답과 얼마나 맞는지"를 사람이 만든 정답지(골든 데이터셋)와 자동으로 비교해서 점수로 보여주는 채점 시스템.

왜 쓰나요?

일반 코드는 "맞다/틀리다"가 명확합니다. [add(2, 3)]은 항상 [5]여야 합니다.
하지만 LLM은 같은 입력에도 출력이 흔들리고, "이 정도면 좋은 답"이라는 기준이 애매합니다.

이걸 측정 안 하면 생기는 문제:
- 프롬프트를 고쳤는데 좋아졌는지 나빠졌는지 감으로만 판단하게 됨
- 모델을 sonnet -> haiku로 바꿨을 때 품질이 떨어졌는지 모르고 배포함
- "AI 정확도 80%"라는 말을 숫자로 증명할 수 없음

Evaluation Harness는 이 "감"을 "숫자"로 바꿉니다.

언제 쓰나요?

- 적합한 상황:
  - 프롬프트/모델을 바꾸기 전후 품질 비교 (A/B)
  - 정답이 정해진 분류 작업 (severity = Critical/Major/Minor)
  - 회귀 테스트 (모델 교체 후 기존 성능 유지 확인)
- 부적합한 상황:
  - 정답이 하나로 안 정해지는 창작 작업 (소설 생성 등) -> 사람 평가나 LLM-as-judge 필요
  - 골든 데이터셋을 만들 수 없는 초기 단계

핵심 개념 3가지

1. 골든 데이터셋 (Golden Dataset)

사람이 직접 만든 "정답지". [[{입력, 기대 정답}, ...]] 형태.
LLM이 만들면 안 됨 — 채점 기준 자체가 흔들리기 때문. 그래서 수동으로 만든다.

2. evaluate() vs 일반 pytest

항목	pytest 단위테스트	LangSmith evaluate()
판정	assert (통과/실패)	score 0~1 (정확도 %)
결과 추세	안 남음	대시보드에 시계열 누적
목적	"터지나 안 터지나"	"얼마나 잘하나"

3. evaluator 함수

[(run, example)]을 받아 [{"score": 0 또는 1}]을 반환하는 채점관 함수.

- [run.outputs]: 우리 함수가 실제로 뱉은 결과
- [example.outputs]: 골든 데이터셋의 정답
- 둘이 같으면 1점, 다르면 0점 -> 전체 평균이 정확도

어떻게 동작하나요? (코드 흐름)

```
[python]
from langsmith import evaluate
from langsmith.schemas import Run, Example

# 1) 골든 데이터셋 — 사람이 만든 정답지
GOLDEN_DATASET = [
    {
        "inputs":  {"message": "f-string으로 SQL 조합", "current_severity": "Major"},
        "outputs": {"expected": "Critical"},   # 사람이 정한 정답
    },
    # ... 5개
]

# 2) 평가 대상 함수 — inputs를 받아 우리 시스템 결과를 반환
def target_function(inputs: dict) -> dict:
    severity = judge_severity(inputs["message"], inputs["current_severity"])
    return {"severity": severity}

# 3) 채점 함수 — run(실제) vs example(정답) 비교
def severity_accuracy(run: Run, example: Example) -> dict:
    predicted = run.outputs.get("severity")         # 우리 함수 출력
    expected  = example.outputs["expected"]          # 골든 정답
    return {"score": int(predicted == expected)}    # 같으면 1, 다르면 0

# 4) 실행 — 5개 입력을 모두 돌려 평균 점수 계산
results = evaluate(
    target_function,
    data=GOLDEN_DATASET,
    evaluators=[severity_accuracy],
    experiment_prefix="dev2-severity-judge",
)
```

흐름 단계:

1. evaluate()가 데이터셋의 각 row의 [inputs]를 [target_function]에 넣음
2. 함수 결과가 [run.outputs]에 담김

3. 정답 [example.outputs]와 함께 [severity_accuracy]로 전달
4. 각 row마다 0/1 -> 5개 평균 = 정확도 (예: 4/5 = 0.8)
5. LangSmith 대시보드에 [dev2-severity-judge-{timestamp}] 실험으로 기록

"score"가 0 또는 1인 이유

[int(True)] = 1, [int(False)] = 0.
한 건당 맞으면 1, 틀리면 0으로 채점하면, 여러 건의 평균이 그대로 정확도 비율이 된다.
(맞은 개수 / 전체 개수 = 평균 score). 부분 점수가 필요하면 0.5 같은 실수도 가능.

3rd-Project-sub2 프로젝트에서

파일: [dev2/tests/eval/golden_dataset.py], [dev2/tests/eval/test_severity_eval.py]

역할:

- [dev2/nodes/llm_judge.py]의 [_judge_single_issue()]가 매기는 severity(Critical/Major/Minor)가 사람이 정한 정답과 얼마나 일치하는지 측정.
- llm_judge는 [{"severity", "suggestion", "rationale"}]를 반환 -> evaluator는 그중 [severity]만 채점.
- CI에서는 [LANGCHAIN_API_KEY]가 없으면 [@pytest.mark.skipif]로 자동 스킵 (CLAUDE.md §6 비용 통제와 동일 철학).
- 정확도 70% 미만이면 테스트 실패 -> 품질 게이팅.

```
[python]
import os
import pytest

@pytest.mark.skipif(
    not os.getenv("LANGCHAIN_API_KEY"),
    reason="LANGCHAIN_API_KEY 없음 — CI에서 LangSmith 평가 스킵",
)
def test_severity_accuracy_above_threshold():
    results = evaluate(target_function, data=GOLDEN_DATASET,
                       evaluators=[severity_accuracy],
                       experiment_prefix="dev2-severity-judge")
    avg = sum(r["evaluation_results"]["results"][0].score
              for r in results) / len(GOLDEN_DATASET)
    assert avg >= 0.7, f"정확도 {avg:.0%} < 70% 임계값"
```

실무에서 왜 중요한가

- 모델 교체 회귀 테스트: sonnet -> haiku 바꿔도 같은 데이터셋으로 점수 비교 -> 안전하게 비용 절감
- 프롬프트 A/B: severity_judge 프롬프트 개선 전후를 같은 실험명으로 비교, 대시보드에서 추세 확인
- 품질 게이팅: 70% 미만이면 머지 차단 (CI 통합)
- 비용 통제: API 키 없는 CI 환경에서는 자동 스킵 -> 평소엔 안 돌고 필요할 때만 실행

복습 퀴즈 (3문항)

1. (이해) evaluator 함수가 받는 [run.outputs]와 [example.outputs]는 각각 무엇을 담고 있나요?
2. (적용) severity 분류가 5개 중 3개 맞았다면 평균 score는 얼마이고, 70% 임계값을 통과하나요?
3. (비교) 일반 pytest의 [assert]와 LangSmith [evaluate()]의 score 방식은 무엇이 다르고, 왜 LLM에는 score 방식이 더 적합한가요?